



Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

Лабораторна робота №8

із дисципліни «Технології Розробки Програмного Забезпечення»

Тема: ШАБЛОНИ «COMPOSITE», «FLYWEIGHT», «INTERPRETER», «VISITOR»

Варіант-28

Виконав:

Студент групи ІА-24

Коханчук Михайло Миколайович

Перевірив:

Мягкий Михайло Юрійович

Київ-2024

ЗМІСТ

Лабораторна робота №8	1
МЕТА	3
Теоритичні відомості:	4
Завдання:.....	17
Хід роботи:	17
<i>Рис. 1. Ієрархія класів. Реалізація патерну Flyweight.</i>	17
<i>Рис. 2. Інтерфейс Metadata для реалізації патерну Flyweight.</i>	17
<i>Рис. 2.1.1 Клас-реалізація інтерфейсу Metadata.</i>	18
<i>Рис. 2.1.2 Клас MetadataFactory для реалізації «Фабрики легковаговиків»</i>	18
<i>Рис. 2.1.3 Клас для JSON-властивостей із унікальними значеннями.</i>	19
<i>Рис. 2.1.4 Клас JsonSchemaManager для роботи з JSON із спільними метаданими</i>	19
ВИСНОВОК	20

META

Розробити функціонал застосунку для роботи з JSON, зокрема забезпечити оптимізацію використання пам'яті при обробці метаданих JSON-схем. Реалізувати та інтегрувати шаблон проектування Flyweight для ефективного управління об'єктами, що мають спільні властивості.

Теоритичні відомості:

Шаблон «COMPOSITE»

Компонувальник — це структурний патерн проектування, що дає змогу згрупувати декілька об'єктів у деревоподібну структуру, а потім працювати з нею так, ніби це одиничний об'єкт.

Проблема

Патерн Компонувальник має сенс тільки в тих випадках, коли основна модель вашої програми може бути структурована у вигляді дерева. Наприклад, є два об'єкти — Продукт і Коробка. Коробка може містити кілька Продуктів та інших Коробок меншого розміру. Останні, в свою чергу, також містять або Продукти, або Коробки і так далі.

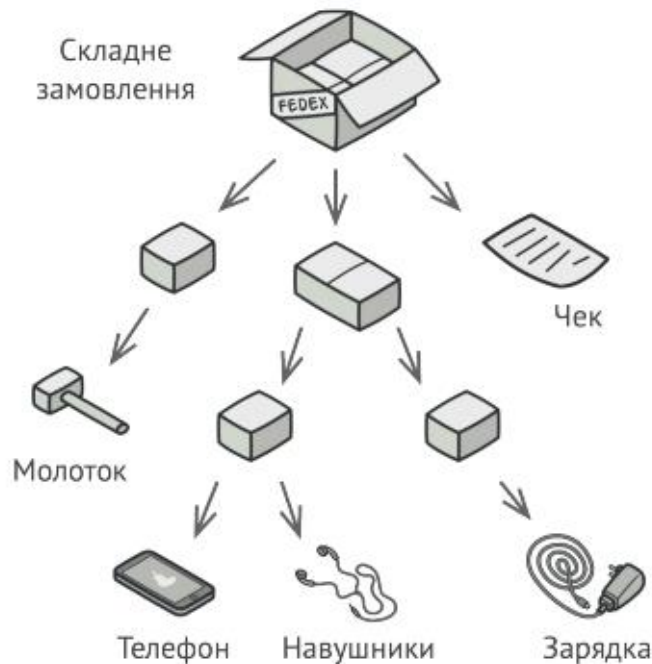


Рис.2.1 Замовлення може складатися з різних продуктів, запакованих у власні коробки

Тепер, припустімо, що ваші Продукти й Коробки можуть бути частиною замовлень. При цьому замовлення може містити як звичайні Продукт без пакування, так і наповнені змістом Коробки. Ваше завдання полягає в тому, щоб дізнатися вартість всього замовлення. Якщо спробувати вирішити завдання напролом, тоді потрібно відкрити усі коробки замовлення, перебрати продукти й порахувати їхню загальну вартість. Але це занадто велика морока, оскільки типи коробок і їхній вміст можуть бути вам невідомі заздалегідь. Крім того, наперед невідомою є і кількість рівнів вкладеності коробок, тому перебрати коробки простим циклом не вийде.

Рішення

Компонувальник пропонує розглядати Продукт і Коробку через єдиний інтерфейс зі спільним методом отримання ціни. Продукт просто поверне свою вартість, а Коробка запитає про вартість кожного предмета всередині себе і поверне суму результатів. Якщо одним із внутрішніх предметів виявиться трохи менша коробка, вона теж буде перебирати власний вміст, і так далі, доки не порахується вміст усіх складових частин.



Рис.2.2 Компонувальник рекурсивно запускає дію по всіх компонентах дерева — від коріння до листя.

Для вас як клієнта важливим є те, що вже не потрібно нічого знати про структуру замовлень. Ви викликаєте метод отримання ціни, він повертає цифру, і ви не «тонете» в горах картону та скотчу.

Структура

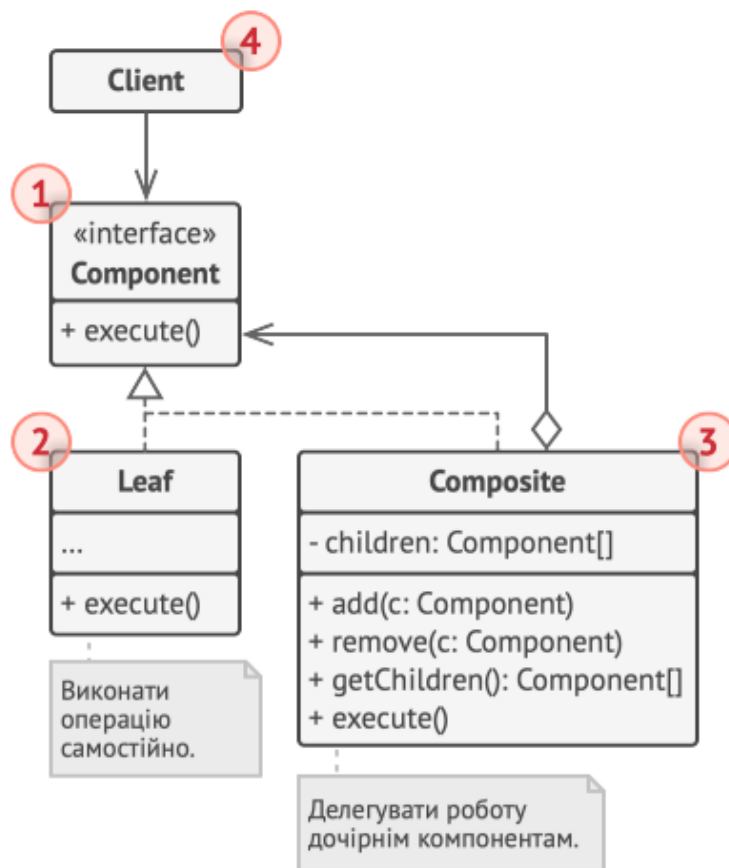


Рис.2.3 Структура патерну Компонувальник

1. Компонент описує загальний інтерфейс для простих і складових компонентів дерева.
2. Лист — це простий компонент дерева, який не має відгалужень. Класи листа міститимуть більшу частину корисного коду, тому що їм нікому передавати його виконання.
3. Контейнер (або композит) — це складовий компонент дерева. Він містить набір дочірніх компонентів, але нічого не знає про їхні типи. Це можуть бути як прості компоненти-листя, так і інші компоненти-контейнери. Проте, це не проблема, якщо усі дочірні компоненти дотримуються єдиного інтерфейсу. Методи контейнера переадресовують основну роботу своїм дочірнім компонентам, хоча можуть додавати щось своє до результату.
4. Клієнт працює з деревом через загальний інтерфейс компонентів. Завдяки цьому, клієнту не важливо, що перед ним знаходиться — простий чи складовий компонент дерева.

Застосування

Якщо вам потрібно представити деревоподібну структуру об'єктів

Патерн Компонувальник пропонує зберігати в складових об'єктах посилання на інші прості або складові об'єкти. Вони, у свою чергу, теж можуть зберігати свої вкладені об'єкти і так далі. У підсумку, ви можете будувати складну деревоподібну структуру даних, використовуючи всього два основних різновиди об'єктів.

Якщо клієнти повинні однаково трактувати прості та складові об'єкти

Завдяки тому, що прості та складові об'єкти реалізують спільний інтерфейс, клієнту байдуже, з яким саме об'єктом він працюватиме

Кроки реалізації

1. Переконайтеся, що вашу бізнес-логіку можна представити як деревоподібну структуру. Спробуйте розбити її на прості компоненти й контейнери. Пам'ятайте, що контейнери можуть містити як прості компоненти, так і інші вкладені контейнери.
2. Створіть загальний інтерфейс компонентів, який об'єднає операції контейнерів та простих компонентів дерева. Інтерфейс буде вдалим, якщо ви зможете використовувати його, щоб взаємозамінити прості й складові компоненти без втрати сенсу.
3. Створіть клас компонентів-листя, які не мають подальших відгалужень. Майте на увазі, що програма може містити декілька таких класів
4. Створіть клас компонентів-контейнерів і додайте до нього масив для зберігання посилань на вкладені компоненти. Цей масив повинен бути здатен містити як прості, так і складові компоненти, тому переконайтеся, що його оголошено з типом інтерфейсу компонентів. Реалізуйте в контейнері методи інтерфейсу компонентів, пам'ятаючи про те, що контейнери повинні делегувати основну роботу своїм дочірнім компонентам
5. Додайте операції додавання й видалення дочірніх компонентів до класу контейнерів. Майте на увазі, що методи додавання/видалення дочірніх компонентів можна оголосити також і в інтерфейсі компонентів. Так, це порушить принцип розділення інтерфейсу, тому що реалізації методів будуть порожніми в компонентах-листях. Проте усі компоненти дерева стануть дійсно однаковими для клієнта.

Переваги

- Спрощує архітектуру клієнта при роботі зі складним деревом компонентів.
- Полегшує додавання нових видів компонентів.

Недоліки

Створює занадто загальний дизайн класів.

Шаблон «FLYWEIGHT»

Легковаговик — це структурний патерн проектування, що дає змогу вмістити більшу кількість об'єктів у відведеній оперативній пам'яті. Легковаговик заощаджує пам'ять, розподіляючи спільний стан об'єктів між собою, замість зберігання однакових даних у кожному об'єкті.

Проблема

На дозвіллі ви вирішили написати невелику гру, в якій гравці переміщуються по карті та стріляють один в одного. Фішкою гри повинна була стати реалістична система частинок. Кулі, снаряди, уламки від вибухів — все це повинно реалістично літати та гарно виглядати.

Гра відмінно працювала на вашому потужному комп'ютері, проте ваш друг повідомив, що гра починає гальмувати й вилітає через кілька хвилин після запуску. Передивившись логи, ви виявили, що гра вилітає через нестачу оперативної пам'яті. У вашого друга комп'ютер значно менше «прокачаний», тому проблема в нього й проявляється так швидко. Дійсно, кожна частинка у грі представлена власним об'єктом, що має безліч даних. У певний момент, коли побоїще на екрані досягає кульмінації, оперативна пам'ять комп'ютера вже не може вмістити нові об'єкти частинок, і програма «вилітає».

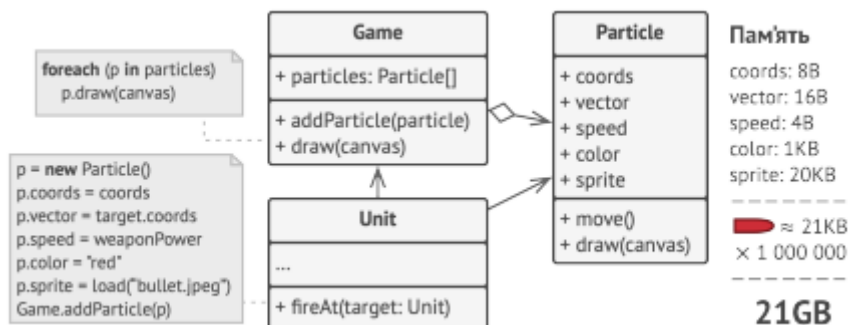


Рис.2.4 Заангажована оперативна пам'ять

Рішення

Якщо уважно подивитися на клас частинок, то можна помітити, що колір і спрайт займають найбільше пам'яті. Більше того, ці поля зберігаються в кожному об'єкті, хоча фактично їхні значення є однаковими для більшості частинок.

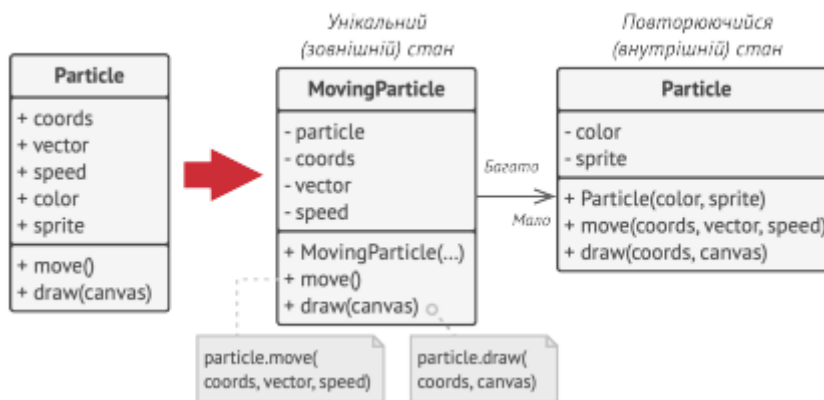


Рис.2.5 Розбиття класу частинок на два класи

Інший стан об'єктів — координати, вектор руху й швидкість відрізняються для всіх частинок. Таким чином, ці поля можна розглядати як контекст, у якому використовується частинка, а колір і спрайт — це дані, що не змінюються в часі. Незмінні дані об'єкта прийнято називати «внутрішнім станом». Всі інші дані — це «зовнішній стан».

Патерн Легковаговик пропонує не зберігати зовнішній стан у класі, а передавати його до тих чи інших методів через параметри. Таким чином, одні і ті самі об'єкти можна буде повторно використовувати в різних контекстах. Головна ж перевага в тому, що тепер знадобиться набагато менше об'єктів, адже вони тепер відрізнятимуться тільки внутрішнім станом, а він не має так багато варіацій.

У нашому прикладі з частинками достатньо буде залишити лише три об'єкти, що відрізняються спрайтами і кольором — для куль, снарядів та уламків. Нескладно здогадатися, що такі полегшені об'єкти називають легковаговиками

Але куди переїде зовнішній стан? Адже хтось повинен його зберігати. Найчастіше його переміщують до контейнера, який керував об'єктами до застосування патерна. В нашому випадку це був головний клас гри. Ви могли б додати до його класу поля-масиви для зберігання координат, векторів і швидкостей частинок. Крім цього, потрібен буде ще один масив для зберігання посилань на об'єкти-легковаговики, що відповідають тій чи іншій частинці.

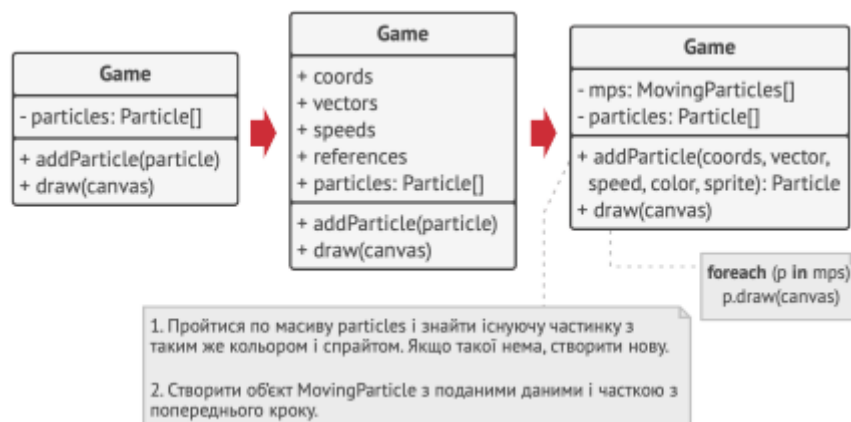


Рис.2.6. Створення класу-контексту для прив'язки зовнішнього стану із легковаговиком

Більш елегантним рішенням було б створити додатковий клас-контекст, який пов'язував би зовнішній стан з тим чи іншим легковаговиком. Це дозволить обійтися тільки одним полем-масивом у класі контейнера. «Але стривайте, нам буде потрібно стільки ж цих об'єктів, скільки було на самому початку!» — скажете ви і будете праві! Але річ у тім, що об'єкти-контексти займають набагато менше місця, ніж початкові. Адже найважчі поля залишилися всередині легковаговика (вибачте за каламбур), і зараз ми будемо посилатися на ці об'єкти з контекстів, замість того, щоб повторно зберігати стан, що дублюється.

Незмінність Легковаговиків

Оскільки об'єкти легковаговиків будуть використані в різних контекстах, ви повинні бути впевненими в тому, що їхній стан неможливо змінити після створення. Весь внутрішній стан легковаговик повинен отримувати через параметри конструктора. Він не повинен мати сеттерів і публічних полів

Фабрика Легковаговиків

Для зручності роботи з легковаговиками і контекстами можна створити фабричний метод, що приймає в параметрах увесь внутрішній (іноді й зовнішній) стан бажаного об'єкта. Найбільша користь цього методу в тому, щоб знаходити вже створених легковаговиків з таким самим внутрішнім станом, як потрібно. Якщо легковаговик знаходиться, його можна повторно використовувати. Якщо немає — просто створюємо новий.

Зазвичай цей метод додають до контейнера легковаговиків або створюють окремий клас-фабрику. Його навіть можна зробити статичним і розмістити в класі легковаговиків.

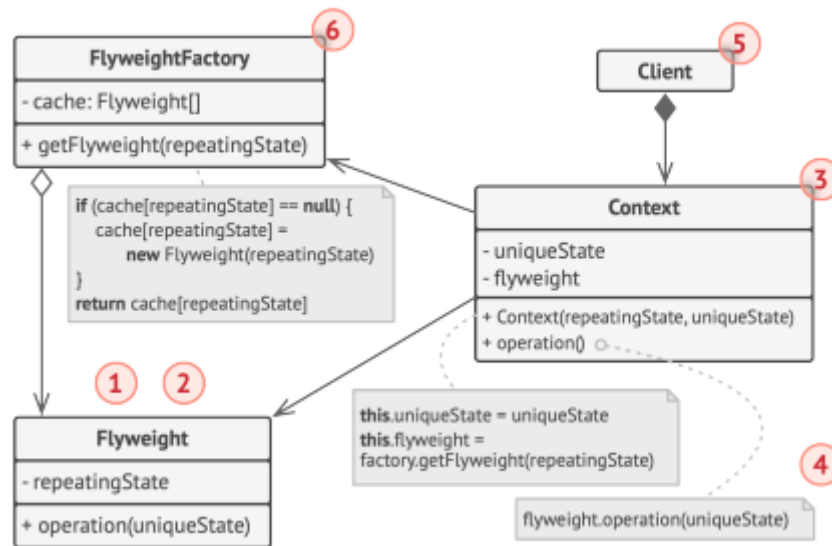


Рис.2.7 Структура патерну Легковаговик-Фабрика

1. Ви завжди повинні пам'ятати про те, що легковаговик застосовується в програмі, яка має величезну кількість однакових об'єктів. Цих об'єктів повинно бути так багато, щоб вони не вміщалися в доступній оперативній пам'яті без додаткових хитрощів. Патерн розділяє дані цих об'єктів на дві частини — легковаговики та контексти.
2. Легковаговик містить стан, який повторювався в багатьох первинних об'єктах. Один і той самий легковаговик може використовуватись у зв'язці з безліччю контекстів. Стан, що зберігається тут, називається внутрішнім, а той, який він отримує ззовні, — зовнішнім.
3. Контекст містить «зовнішню» частину стану, унікальну для кожного об'єкта. Контекст пов'язаний з одним з об'єктів-легковаговиків, що зберігають стан, який залишився.
4. Поведінку оригінального об'єкта найчастіше залишають у легковаговику, передаючи значення контексту через параметри методів. Тим не менше, поведінку можна розмістити й в контексті, використовуючи легковаговик як об'єкт даних
5. Клієнт обчислює або зберігає контекст, тобто зовнішній стан легковаговиків. Для клієнта легковаговики виглядають як шаблонні об'єкти, які можна налаштувати під час використання, передавши контекст через параметри
6. Фабрика легковаговиків керує створенням і повторним використанням легковаговиків. Фабрика отримує запити, в яких зазначено бажаний стан легковаговика. Якщо легковаговик з таким станом вже створений, фабрика відразу його повертає, а якщо ні — створює новий об'єкт

Застосування

Якщо не вистачає оперативної пам'яті для підтримки всіх потрібних об'єктів.

Ефективність патерна Легковаговик багато в чому залежить від того, як і де він використовується.

Застосовуйте цей патерн у випадках, коли виконано всі перераховані умови:

- у програмі використовується велика кількість об'єктів;
- через це високі витрати оперативної пам'яті;
- більшу частину стану об'єктів можна винести за межі їхніх класів;
- великі групи об'єктів можна замінити невеликою кількістю об'єктів, що розділяються, оскільки зовнішній стан винесено.

Кроки реалізації

1. Розділіть поля класу, який стане легковаговику, на дві частини:
 - внутрішній стан: значення цих полів однакові для великої кількості об'єктів.
 - зовнішній стан (контекст): значення полів унікальні для кожного об'єкта.
2. Залишіть поля внутрішнього стану в класі, але переконайтеся, що їхні значення неможливо змінити. Ці поля повинні ініціалізуватись тільки через конструктор.

3. Перетворіть поля зовнішнього стану на параметри методів, у яких ці поля використовувалися. Потім видаліть поля з класу.
4. Створіть фабрику, яка буде кешувати та повторно віддавати вже створені об'єкти. Клієнт повинен отримувати легковаговика з певним внутрішнім станом саме з цієї фабрики, а не створювати його безпосередньо.
5. Клієнт повинен зберігати або обчислювати значення зовнішнього стану (контекст) і передавати його до методів об'єкта легковаговика.

Переваги

Заощаджує оперативну пам'ять.

Недоліки

- Витрачає процесорний час на пошук/обчислення контексту.
- Ускладнює код програми внаслідок введення безлічі додаткових класів.

Шаблон «INTERPRETER»

Інтерпретатор — шаблон проектування, належить до класу шаблонів поведінки. У разі, якщо якась задача виникає досить часто, є сенс подати її конкретні проявлення у вигляді речень простою мовою. Потім можна буде створити інтерпретатор, котрий вирішує задачу, аналізуючи речення цієї мови. Наприклад, пошук рядків за зразком — досить розповсюджена задача. Регулярні вирази — це стандартна мова для задання зразків пошуку.

Проблема

Стоїть задача пошуку рядків по зразку, як така, що часто зустрічається. Або ж загалом є якась задача, яка дуже часто змінюється.

Рішення

Може бути вирішена шляхом створення інтерпретатора, який визначає граматику мови. "Клієнт" будує речення у вигляді абстрактного синтаксичного дерева, у вузлах якого знаходяться об'єкти класів "НетермінальнийВираз" і "ТермінальнийВираз" (рекурсивне), потім "Клієнт" ініціалізує контекст і виражає операцію Розібрати(Контекст). На кожному вузлі типу "НетермінальнийВираз" визначається операція Розібрати для кожного підвиразу. Для класу "ТермінальнийВираз" операція Розібрати визначає базу рекурсії. "АбстрактнийВираз" визначає абстрактну операцію Розібрати, загальну для всіх вузлів в абстрактному синтаксичному дереві. "Контекст" містить інформацію, глобальну по відношенню до інтерпретатора.

Приклад з життя:

Припустимо потрібний автоматичний розбір поставленого завдання таким чином, що воно розбивається на прості та стає зрозумілим системі.

Структура

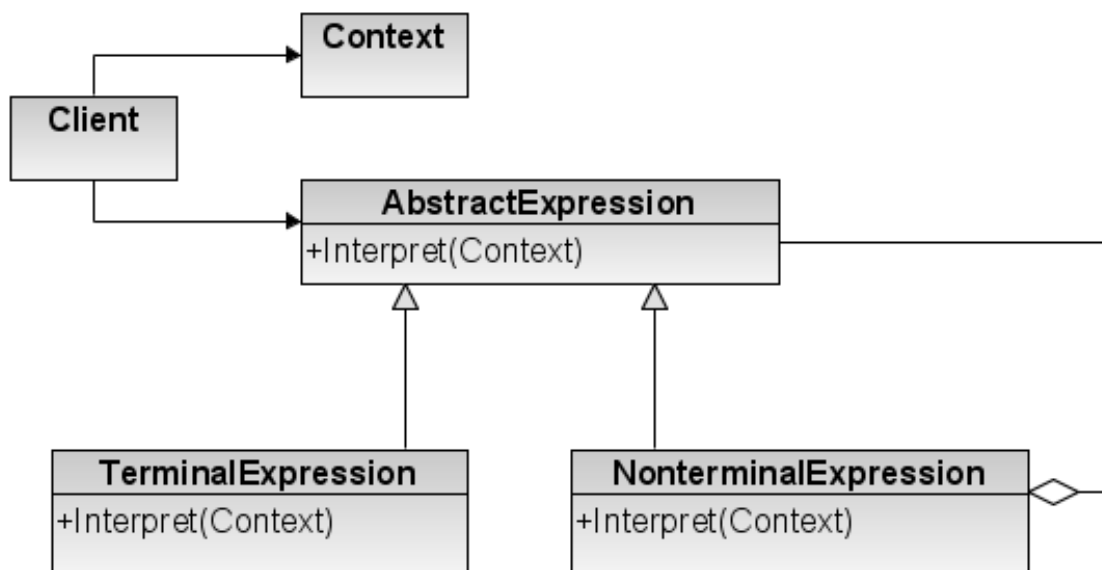


Рис.2.8 Структура патерну Інтерпретатор

Застосування

Даний шаблон використовується для подання граматики і інтерпретатора для вибраної мови (наприклад, скриптової). Граматика мови представлена термінальними і нетермінальними символами, кожен з яких інтерпретується в контексті використання. Клієнт передає контекст і сформовану пропозицію в використовувану мову в термінах абстрактного синтаксичного дерева (деревоподібна структура, яка однозначно визначає ієрархію виклику підвиразів), кожен вираз інтерпретується окремо з використанням контексту. У разі наявності дочірніх виразів, батьківський вираз інтерпретує спочатку дочірні (рекурсивно), а потім обчислює результат власної операції.

Шаблон зручно використовувати в разі невеликої граматики (інакше розростеться кількість використовуваних класів) і відносно простого контексту (без взаємозалежностей і т.п.).

Даний шаблон визначає базовий каркас інтерпретатора, який за допомогою рекурсії повертає результат обчислення пропозиції на основі результатів окремих елементів.

При використанні даного шаблону дуже легко реалізовується і розширюється граматика, а також додаються нові способи інтерпретації виразів

Переваги

- Граматику стає легко розширювати та змінювати, реалізації класів, що описують вузли абстрактного синтаксичного дерева схожі (легко кодуються).
- Можна легко змінювати спосіб обчислення виразів.

Недоліки

Супровід граматики з великою кількістю правил важко.

Шаблон «VISITOR»

Відвідувач — це поведінковий патерн проектування, що дає змогу додавати до програми нові операції, не змінюючи класи об'єктів, над якими ці операції можуть виконуватися.

Проблема

Ваша команда розробляє програму, що працює з гео даними у вигляді графа. Вузлами графа можуть бути як міста, так інші локації, такі, як пам'ятки, великі підприємства тощо. Кожен вузол має посилання на найближчі до нього вузли. Для кожного типу вузла існує свій власний клас, а кожен вузол представлений окремим об'єктом



Рис.2.9 Експорт гео-вузлів до XML.

Ваше завдання — зробити експорт цього графа до XML. Справа була б легкою, якщо б ви могли редагувати класи вузлів. У цьому випадку можна було б додати метод експорту до кожного типу вузлів, а потім, перебираючи всі вузли графа, викликати цей метод для кожного вузла. Завдяки поліморфізму, рішення було б елегантним, оскільки ви могли б не прив'язуватися до конкретних класів вузлів. Але, на жаль, змінити класи вузлів у вас не вийшло. Системний архітектор сказав, що код класів вузлів зараз дуже стабільний, і від нього багато що залежить, а тому він не хоче ризикувати, дозволяючи будь-кому чіпати цей код.

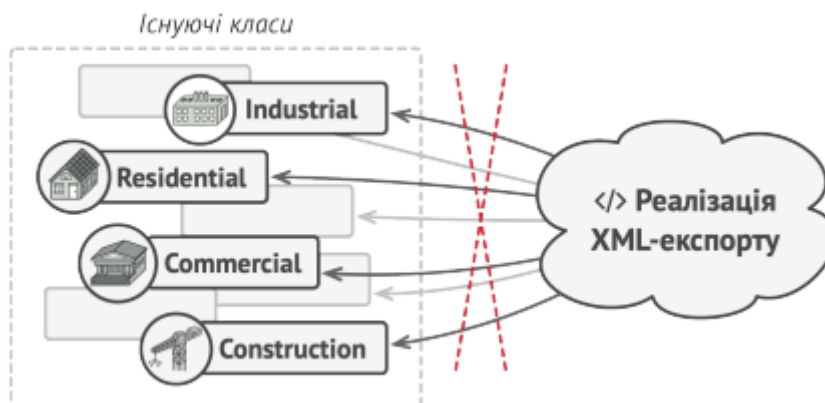


Рис.2.10 Код XML-експорту доведеться додати до всіх класів вузлів, а це дуже невигідно.

До того ж він сумнівався в тому, що експорт до XML взагалі є доречним в рамках цих класів. Їхнє основне завдання пов'язане з гео даними, а експорт виглядає в межах цих класів, як біла ворона. Була ще одна причина заборони. Наступного тижня вам міг знадобитися експорт в який-небудь інший формат даних, а це призвело б до повторних змін в класах.

Рішення

Патерн Відвідувач пропонує розмістити нову поведінку в окремому класі, замість того, щоб множити її відразу в декількох класах. Об'єкти, з якими повинна бути пов'язана поведінка, не виконуватимуть її самостійно. Замість цього ви будете передавати ці об'єкти до методів відвідувача.

Код поведінки, імовірно, повинен відрізнятися для об'єктів різних класів, тому й методів у відвідувача повинно бути декілька. Назви та принцип дії цих методів будуть подібними, а основна відмінність торкатиметься типу, що приймається в параметрах об'єкта, наприклад:

```
1 class ExportVisitor implements Visitor is
2     method doForCity(City c) { ... }
3     method doForIndustry(Industry f) { ... }
4     method doForSightSeeing(SightSeeing ss) { ... }
5     // ...
```

Рис.2.11 Розміщення нової поведінки в окремому класі

Тут виникає запитання, яким чином ми будемо подавати вузли до об'єкта відвідувача. Оскільки усі методи відрізняються сигнатурою, використати поліморфізм при перебиранні вузлів не вийде. Доведеться перевіряти тип вузлів для того, щоб вибрати відповідний метод відвідувача.

```
1 foreach (Node node : graph)
2     if (node instanceof City)
3         exportVisitor.doForCity((City) node);
4     if (node instanceof Industry)
5         exportVisitor.doForIndustry((Industry) node);
6     // ...
```

Рис.2.12 Перевірка типів вузлів

Тут не допоможе навіть механізм перевантаження методів (доступний у Java і C#). Якщо назвати всі методи однаково, то невизначеність реального типу вузла все одно не дасть викликати правильний метод. Механізм перевантаження весь час викликатиме метод відвідувача, відповідний типу Node, а не реального класу поданого вузла.

Але патерн Відвідувач вирішує і цю проблему, використовуючи механізм подвійної диспетчеризації. Замість того, щоб самим шукати потрібний метод, ми можемо доручити це об'єктам, які передаємо в параметрах відвідувачеві, а вони вже самостійно викличуть правильний метод відвідувача.

```
1 // Client code
2 foreach (Node node : graph)
3     node.accept(exportVisitor);
4
5 // City
6 class City is
7     method accept(Visitor v) is
8         v.doForCity(this);
9     // ...
10
11 // Industry
12 class Industry is
13     method accept(Visitor v) is
14         v.doForIndustry(this);
15     // ...
```

Рис.2.13 Механізм подвійної диспетчеризації

Як бачите, змінити класи вузлів все-таки доведеться. Проте ця проста зміна дозволить застосувати до об'єктів вузлів й інші поведінки, адже класи вузлів будуть прив'язані не до конкретного класу відвідувачів, а до їхнього загального інтерфейсу. Тому, якщо доведеться додати до програми нову поведінку, ви створите новий клас відвідувачів і будете передавати його до методів вузлів.

Аналогія з життя

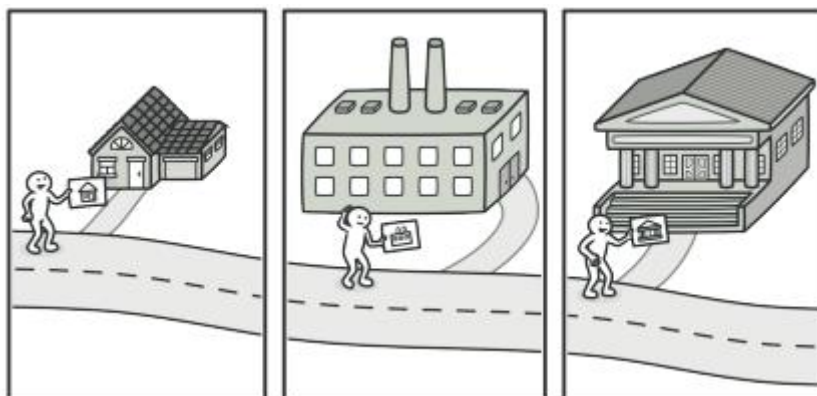


Рис.2.14 У страхового агента приготовані поліси для різних видів організацій

Уявіть собі страхового агента-початківця, який прагне отримати нових клієнтів. Він хаотично відвідує всі будинки навколо, пропонуючи свої послуги. Але для кожного типу будинків, які він відвідує, у нього є особлива пропозиція.

- Прийшовши до будинку звичайної сім'ї, він пропонує оформити медичну страховку
- Прийшовши до банку, він пропонує страховку на випадок пограбування
- Прийшовши на фабрику, він пропонує страхування підприємства на випадок пожежі чи повені

Структура

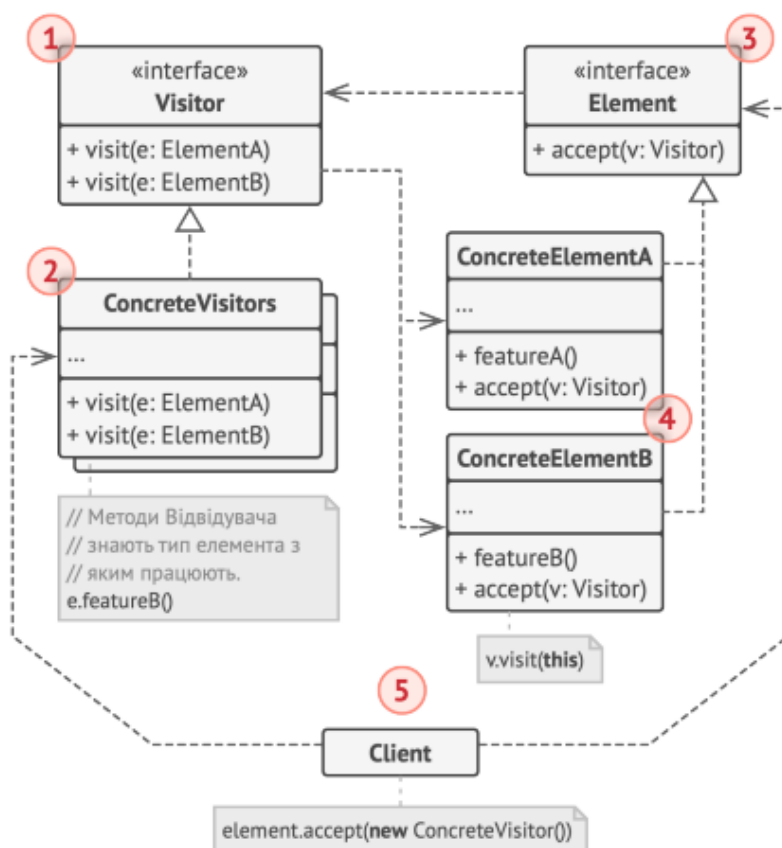


Рис.2.15 Структура патерну Відвідувач

1. Відвідувач описує спільний для всіх типів відвідувачів інтерфейс. Він оголошує набір методів, що відрізняються типом вхідного параметра. Кожному класу конкретних елементів повинен підходити свій метод. В мовах, які підтримують перевантаження методів, ці методи можуть мати однакові імена, але типи їхніх параметрів повинні відрізнятися.
2. Конкретні відвідувачі реалізують якусь особливу поведінку для всіх типів елементів, які можна подати через методи інтерфейсу відвідувача.
3. Елемент описує метод прийому відвідувача. Цей метод повинен мати лише один параметр, оголошений з типом загального інтерфейсу відвідувачів
4. Конкретні елементи реалізують методи приймання відвідувача. Мета цього методу — викликати той метод відвідування, який відповідає типу цього елемента. Так відвідувач дізнається, з яким типом елемента він працює.
5. Клієнтом зазвичай виступає колекція або складний складовий об'єкт, наприклад, дерево Компонувальника. Здебільшого, клієнт не прив'язаний до конкретних класів елементів, працюючи з ними через загальний інтерфейс елементів.

Застосування

Якщо вам потрібно виконати якусь операцію над усіма елементами складної структури об'єктів, наприклад, дерево

Відвідувач дозволяє застосовувати одну і ту саму операцію до об'єктів різних класів.

Якщо над об'єктами складної структури об'єктів потрібно виконувати деякі не пов'язані між собою операції, але ви не хочете «засмічувати» класи такими операціями.

Відвідувач дозволяє витягти споріднені операції з класів, що складають структуру об'єктів, помістивши їх до одного класу-відвідувача. Якщо структура об'єктів використовується в декількох програмах, то патерн дозволить кожній програмі мати тільки потрібні в ній операції.

Якщо нова поведінка має сенс тільки для деяких класів з існуючої ієрархії.

Відвідувач дозволяє визначити поведінку тільки для цих класів, залишивши її порожньою для всіх інших.

Кроки реалізації

1. Створіть інтерфейс відвідувача й оголосіть у ньому методи «відвідування» для кожного класу елемента, який існує в програмі.
2. Опишіть інтерфейс елементів. Якщо ви працюєте з уже існуючими класами, оголосіть абстрактний метод прийняття відвідувачів у базовому класі ієрархії елементів.
3. Реалізуйте методи прийняття в усіх конкретних елементах. Вони повинні переадресовувати виклики тому методу відвідувача, в якому тип параметра збігається з поточним класом елемента.
4. Ієрархія елементів повинна знати тільки про загальний інтерфейс відвідувачів. З іншого боку, відвідувачі знатимуть про всі класи елементів.
5. Для кожної нової поведінки створіть свій власний конкретний клас. Пристосуйте цю поведінку для роботи з усіма наявними типами елементів, реалізавши всі методи інтерфейсу відвідувачів. Ви можете зіткнутися з ситуацією, коли відвідувачу потрібен доступ до приватних полів елементів. У цьому випадку ви можете або розкрити доступ до цих полів, порушивши інкапсуляцію елементів, або зробити клас відвідувача вкладеним в клас елемента, якщо вам пощастило писати мовою, яка підтримує механізм вкладених класів.
6. Клієнт створюватиме об'єкти відвідувачів, а потім передаватиме їх елементам через метод прийняття

Переваги

- Спрощує додавання операцій, працюючих зі складними структурами об'єктів.
- Об'єднує споріднені операції в одному класі.

- Відвідувач може накопичувати стан при обході структури елементів

Недоліки

- Патерн невиправданий, якщо ієрархія елементів часто змінюється.
- Може призвести до порушення інкапсуляції елементів.

Завдання:

1. Ознайомитися з короткими теоретичними відомостями.
2. Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.
3. Застосування одного з розглянутих шаблонів при реалізації програми

Хід роботи:

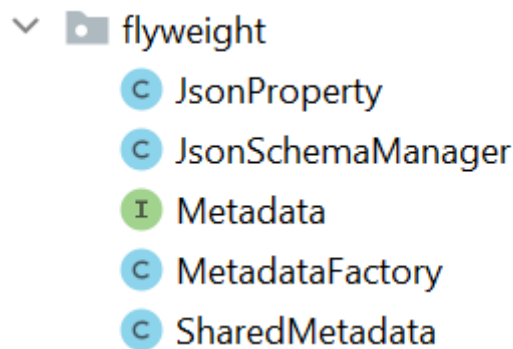


Рис. 1. Ієрархія класів. Реалізація патерну Flyweight.

```
public interface Metadata {  
    no usages 1 implementation mykh3398  
    String getType();  
    1 implementation mykh3398  
    String getFormat();  
    2 usages 1 implementation mykh3398  
    String getDescription();  
}
```

Рис. 2. Інтерфейс Metadata для реалізації патерну Flyweight.

```

public class SharedMetadata implements Metadata {
    2 usages
    private final String type;
    2 usages
    private final String format;
    2 usages
    private final String description;

    1 usage  mykh3398
    public SharedMetadata(String type, String format, String description) {
        this.type = type;
        this.format = format;
        this.description = description;
    }

    no usages  mykh3398
    @Override
    public String getType() { return type; }

    mykh3398
    @Override
    public String getFormat() { return format; }

    2 usages  mykh3398
    @Override
    public String getDescription() { return description; }
}

```

Рис. 2.1.1 Клас-реалізація інтерфейсу Metadata

```

public class MetadataFactory {
    3 usages
    private final Map<String, SharedMetadata> metadataPool = new HashMap<>();

    2 usages  mykh3398
    public SharedMetadata getMetadata(String type, String format, String description) {
        String key = type + ":" + format + ":" + description;
        if (!metadataPool.containsKey(key)) {
            metadataPool.put(key, new SharedMetadata(type, format, description));
        }
        return metadataPool.get(key);
    }
}

```

Рис. 2.1.2 Клас MetadataFactory для реалізації «Фабрики легковаговиків»

```

public class JsonProperty {
    2 usages
    private final String name;
    2 usages
    private final Object value;
    2 usages
    private final SharedMetadata metadata;

    2 usages  ⓘ mykh3398
    public JsonProperty(String name, Object value, SharedMetadata metadata) {
        this.name = name;
        this.value = value;
        this.metadata = metadata;
    }

    2 usages  ⓘ mykh3398
    public String getName() { return name; }

    2 usages  ⓘ mykh3398
    public Object getValue() { return value; }

    2 usages  ⓘ mykh3398
    public SharedMetadata getMetadata() { return metadata; }
}

```

Рис. 2.1.3 Клас для JSON-властивостей із унікальними значеннями

```

public class JsonSchemaManager {
    2 usages
    private final MetadataFactory metadataFactory = new MetadataFactory();

    no usages  ⓘ mykh3398
    public void processSchema() {
        SharedMetadata metadata1 = metadataFactory.getMetadata( type: "string", format: "email", description: "An email address");
        SharedMetadata metadata2 = metadataFactory.getMetadata( type: "integer", format: "none", description: "An integer value");

        JsonProperty property1 = new JsonProperty( name: "email", value: "example@example.com", metadata1);
        JsonProperty property2 = new JsonProperty( name: "age", value: 30, metadata2);

        System.out.println(property1.getName() + ": " + property1.getValue() + " (" + property1.getMetadata().getDescription());
        System.out.println(property2.getName() + ": " + property2.getValue() + " (" + property2.getMetadata().getDescription());
    }
}

```

Рис. 2.1.4 Клас JsonSchemaManager для роботи з JSON із спільними метаданими

Дана реалізація дозволяє оптимізувати роботу із повторюваними метаданими у JSON-схемах. Наприклад, метадані для полів схеми (типи, описи) можуть бути спільними для багатьох полів, і замість дублювання у пам'яті ми зберігаємо їх як об'єкти-легковаговики.

ВИСНОВОК

У ході виконання роботи було реалізовано шаблон проектування Flyweight для оптимізації обробки метаданих JSON-схем. Шаблон дозволив зменшити використання пам'яті шляхом поділу спільного стану об'єктів і зосередження унікального стану в зовнішньому середовищі. Це забезпечило ефективне управління великою кількістю об'єктів, що є актуальним для обробки великих JSON-схем із численними метаданими. Реалізований підхід підвищує продуктивність застосунку та сприяє масштабованості системи, що робить його адаптованим до розширення функціональності в майбутньому.